Checkpoint 3



🎯 Objetivo da Avaliação

Esta avaliação vai verificar sua capacidade de **aplicar os conceitos de POO** aprendidos nas aulas 1 a 9 em um projeto prático. Você deverá entregar um repositório GitHub contendo todos os desafios implementados e um README reflexivo demonstrando seu aprendizado.

📦 Estrutura da Entrega

Requisitos Obrigatórios

1. **Repositório GitHub** com nome padrão: `poo-[seu-nome]`
2. **Commits organizados** por aula (um commit por desafio mínimo)
3. **README.md** completo com respostas às perguntas de reflexão
4. **Código fonte** de todos os desafios implementados

✅ Checklist de Commits por Aula

Aula 1 - Classes e Objetos

- [ ] **Commit:** `feat: aula1 - cria primeira classe com atributos e tipo`
- [ ] Código-fonte de uma classe de objeto
- [ ] Classe principal com teste

Aula 2 - Métodos

- [ ] **Commit:** `feat: aula2 - adiciona metodos com parametros e retorno`

- [ ] Métodos implementados na classe principal
- [ ] Teste dos métodos no `main`
- [ ] Validação de entrada implementada

Aula 3 - Encapsulamento

- [ ] **Commit:** `feat: aula3 - aplica encapsulamento com modificadores de acesso`
- [ ] Atributos com modificadores de acesso
- [ ] Getters e Setters implementados
- [ ] Validação de dados nos setters

Aula 4 - Construtores

- [ ] **Commit:** `feat: aula4 - cria construtores para inicializar objetos`
- [ ] Construtor padrão e customizado
- [ ] Uso de `this()` e `this.`

Aula 5 - Associação

- [ ] **Commit:** `feat: aula5 - cria classe associada a outra classe`
- [ ] Nova classe criada
- [ ] Associação entre classes (TEM-UM)
- [ ] Teste de criação de objeto associado

Aula 6 - Herança

- [ ] **Commit:** `feat: aula6 - cria heranca entre classe mae e classes filhas`
- [ ] Classe mãe criada
- [ ] Classes filhas criadas
- [ ] Uso de `extends` e `super()`
- [ ] Atributo com modificador de acesso adequado

Aula 7 - Polimorfismo

- [ ] **Commit:** `feat: aula7 - implementa polimorfismo com sobrescrita de metodo`
- [ ] Método na superclasse
- [ ] Sobrescrita nas subclasses com `@Override`
- [ ] Teste polimórfico no `main`

Aula 8 - Classes Abstratas

- [ ] **Commit:** `feat: aula8 - transforma classe mae em abstrata`
- [ ] Classe mãe como `abstract class`
- [ ] Método abstrato criado
- [ ] Implementação obrigatória nas filhas

Aula 9 - Interfaces

- [ ] **Commit:** `feat: aula9 - cria interface com metodo abstrato`
- [ ] Interface criada
- [ ] Classes implementando interface (`implements`)
- [ ] Constantes de interface

📝 README.md - Template Obrigatório

Copie este template e responda TODAS as perguntas no seu README.md:

```
# Projeto FiapRide - [Seu Nome Completo]

## 📊 Informações do Aluno

- **Nome:** [Seu Nome Completo]
- **RM:** [Seu RM]
- **Turma:** [Sua Turma]
- **Curso:** [Seu Curso]
- **GitHub:** [@seu-usuario]

---

## 🎯 Descrição do Projeto

Este projeto é o resultado do aprendizado nas aulas 1-9 de Programação Orientada a Objetos, onde desenvolvemos o sistema ........

---

## ✅ Checklist de Implementação

- [x] Aula 1 - Classes e Objetos
- [x] Aula 2 - Métodos
- [x] Aula 3 - Encapsulamento
- [x] Aula 4 - Construtores
- [x] Aula 5 - Associação
- [x] Aula 6 - Herança
- [x] Aula 7 - Polimorfismo
- [x] Aula 8 - Classes Abstratas
- [x] Aula 9 - Interfaces

---

## 🤔 Perguntas de Reflexão

### Aula 1 - Classes e Objetos

**Pergunta:** "**Pergunta:** Por que precisamos criar uma classe `Passageiro`? Não seria mais fácil apenas criar variáveis soltas no main, como `String nomeAna = "Ana"` e `double saldoAna = 50.0`?

**Pense:** E se o FiapRide tiver 1 milhão de usuários? Como a Classe ajuda a resolver isso?"

**Sua Resposta:**
[Escreva sua reflexão aqui - 5-10 linhas]

---

### Aula 2 - Métodos
```

```
**Pergunta:** "Se nós podemos simplesmente fazer `passageiro.saldo = passageiro.saldo \+ 100` di
retamente no código principal, por que dá tanto trabalho criar um método específico chamado `adi
cionarSaldo\(valor\)` para fazer isso? Quais seriam os riscos para a nossa startup de mobilidade
se deixássemos qualquer programador alterar o saldo diretamente?"

**Sua Resposta:**
[Escreva sua reflexão aqui]

---

### Aula 3 - Encapsulamento

**Pergunta:** "No nosso código, os atributos são `private`, mas os métodos `getSaldo\(\)` e `get
Nome\(\)` são `public`. Por que é seguro deixar o `get` público, mas perigoso deixar o atributo
original público?

Pense bem: Qual a diferença entre dar a alguém uma CÓPIA de um documento seu, e entregar o docum
ento ORIGINAL para a pessoa rasurar? "

**Sua Resposta:**
[Escreva sua reflexão aqui]

---

### Aula 4 - Construtores

**Pergunta:** "> "Na nossa classe `Veiculo`, nós tomamos duas decisões arquitetônicas muito impo
rtantes:
>
> 1. Nós **não** criamos o método `setModelo\(\)`.
> 2. O `setPlaca\(\)` foi criado como **privado**, e criamos um método público chamado `atualiza
rPlaca\(\)` para acessá-lo.
>
> Pensando no mundo real e no Clean Code: Por que é um erro gravíssimo clicar em 'Gerar Getters
e Setters para tudo' automaticamente na sua IDE? Como as nossas duas decisões acima protegem o s
istema de fraudes e falhas de lógica?"

_Dica: Pense sobre o que pode ou não mudar fisicamente em um carro, e a diferença entre "alterar
um dado no banco" e "executar um processo real no Detran"_"

**Sua Resposta:**
[Escreva sua reflexão aqui]

---

### Aula 5 - Associação

**Pergunta:** ""No construtor da `Viagem`, nós exigimos o objeto inteiro \(`Passageiro solicitan
te`\). Se o nosso resumo só precisa imprimir o nome da pessoa, não seria mais fácil e mais leve
pedir apenas a String do nome no construtor da Viagem \(`String nomeDoPassageiro`\) em vez do ob
jeto todo?"
```

```
_Pense nas regras de negócio: O que acontece na hora que a Viagem acaba e o sistema precisa desc
ontar o saldo? Se a Viagem tiver apenas a String "Ana Silva", ela consegue mexer no dinheiro del
a? _"

**Sua Resposta:**
[Escreva sua reflexão aqui]

---

### Aula 6 - Herança

**Pergunta:** ""No nosso código, a mãe `Veiculo` possui os atributos `placa` e `modelo` como `pr
ivate`. Quando o `Carro` herda de `Veiculo`, ele recebe esses atributos, mas o código dentro de
`Carro` NÃO consegue fazer `this.placa = "ABC"`. Ele é obrigado a usar o `super\(\)` ou o `setPl
aca\(\)`.

Por que o Java não deixa a filha alterar as variáveis privadas da mãe diretamente? Qual o princí
pio das aulas passadas que isso está protegendo?""

**Sua Resposta:**
[Escreva sua reflexão aqui]

---

### Aula 7 - Polimorfismo

**Pergunta:** "No nosso loop `for \(Veiculo veiculo : frota\)`, a variável `veiculo` é do tipo g
enérico `Veiculo`. Se esquecêssemos de criar o método `calcularAutonomia\(\)` lá na classe mãe `
Veiculo`, nós conseguiríamos chamá-lo dentro do loop, mesmo sabendo que ele existe dentro do `Ca
rro` e da `Moto`? Por que o contrato precisa existir na base da hierarquia?"

**Sua Resposta:**
[Escreva sua reflexão aqui]

---

### Aula 8 - Classes Abstratas

**Pergunta:** "Pense no mundo real: Faz sentido existir um objeto que é APENAS 'Veículo' sem ser
um tipo específico? Você já entrou em uma concessionária e comprou "um veículo" genérico, sem se
r carro, moto, caminhão ou nada disso?

Por que, então, no código, precisamos EXPLICITAMENTE dizer ao Java que `Veiculo` é `abstract`? P
or que ele não deduz isso sozinho?

Pense: Se esquecermos de colocar `abstract`, qual o risco que corremos? Alguém pode criar `new V
eiculo\(\)` e quebrar a lógica do nosso sistema?"

**Sua Resposta:**
[Escreva sua reflexão aqui]

---
```

```
### Aula 9 - Interfaces

**Pergunta:** "Por que Java permite herança simples \(apenas uma mãe\), mas múltipla implementaç
ão de interfaces \(vários contratos\)?

Pense: Se `CarroEletrico` pudesse herdar de `Veiculo` E de `Bateria` ao mesmo tempo \(herança mú
ltipla\), o que aconteceria se AMBAS as mães tivessem um método chamado `ligar\(\)`?

Como as interfaces resolvem esse problema? "

**Sua Resposta:**
[Escreva sua reflexão aqui]

---

## 🚀 Desafios Técnicos Implementados

### Desafio Pessoal (Seu Projeto)

**Qual foi o domínio que você escolheu para seu projeto pessoal?**
[Responda aqui: ex: Sistema de Biblioteca, Playlist de Música, PetShop, etc.]

**Quais classes você criou?**
[Liste suas classes: ex: Livro, Usuario, Emprestimo]

**Qual foi o maior desafio técnico que você enfrentou?**
[Descreva o desafio e como você o resolveu - 10-15 linhas]

---

## 🎓 Conclusão

**O que você aprendeu nestas 9 aulas?**
[Reflexão final - conte sobre sua jornada de aprendizado]

**Qual conceito foi mais difícil de entender?**
[Qual conceito e como você superou a dificuldade?]

**O que você melhoraria no seu projeto se pudesse refazer?**
[Reflexão crítica sobre seu próprio código]
```

📊 Critérios de Avaliação

Pontuação por Categoria

Categoria	Pontos	Critério
Commits Organizados	1.0	Um commit por aula mínimo, mensagens claras
Código Fonte	3.0	Todos os desafios implementados e funcionando

---

Categoria	Pontos	Critério
Reflexão	4.0	Respostas completas e bem elaboradas
Projeto Pessoal	2.0	Implementado e documentado
TOTAL	**10.0**	

🚀 Como Entregar

Passo a Passo

1. **Criar Repositório:**

```
# No GitHub, crie repositório: poo-fiapride-[seu-nome]
```

1. **Clonar Localmente:**

```
git clone https://github.com/seu-usuario/poo-fiapride-seu-nome.git
cd poo-fiapride-seu-nome
```

1. **Estruturar Projeto:**

```
poo-fiapride-seu-nome/
├── src/
│   └── br/
│       └── com/
│           └── fiapride/
│               └── model/
│                   ├── .....java
│                   ├── .....java
│                   ├── .....java
│                   ├── .....java
│                   └── ...
├── README.md
└── .gitignore
```

1. **Commits por Aula:**

```
git add .
git commit -m "feat: aula1 - cria classe"
git push origin main

# Repita para cada aula
```

1. **Preencher README.md:**

- Copie o template acima
- Responda TODAS as perguntas
- Adicione prints dos diagramas

1. **Entregar:**

- Enviar link do repositório para o professor

- Confirmar que o repositório é público

💡 Dicas de Sucesso

Para Ótima Reflexão

1. **Seja sincero:** Admita o que foi difícil
2. **Conecte conceitos:** Relacione com seu projeto pessoal
3. **Use exemplos:** Ilustre com código do seu projeto
4. **Pense crítico:** Questione as decisões de design
5. **Mostre evolução:** Como você mudou de opinião durante as aulas

Para Código Limpo

1. **Convenções Java:** PascalCase para classes, camelCase para métodos
2. **Comentários:** Explique o "por quê", não o "o quê"
3. **Nomes significativos:** Evite `a`, `x`, `tempo`
4. **Valide entradas:** Não aceite valores inválidos

> "A melhor forma de aprender programação é PROGRAMANDO.
>
> Esta avaliação não é sobre decorar conceitos, mas sobre aplicar.
>
> Boa sorte, e divirta-se criando!"

Bons estudos!

